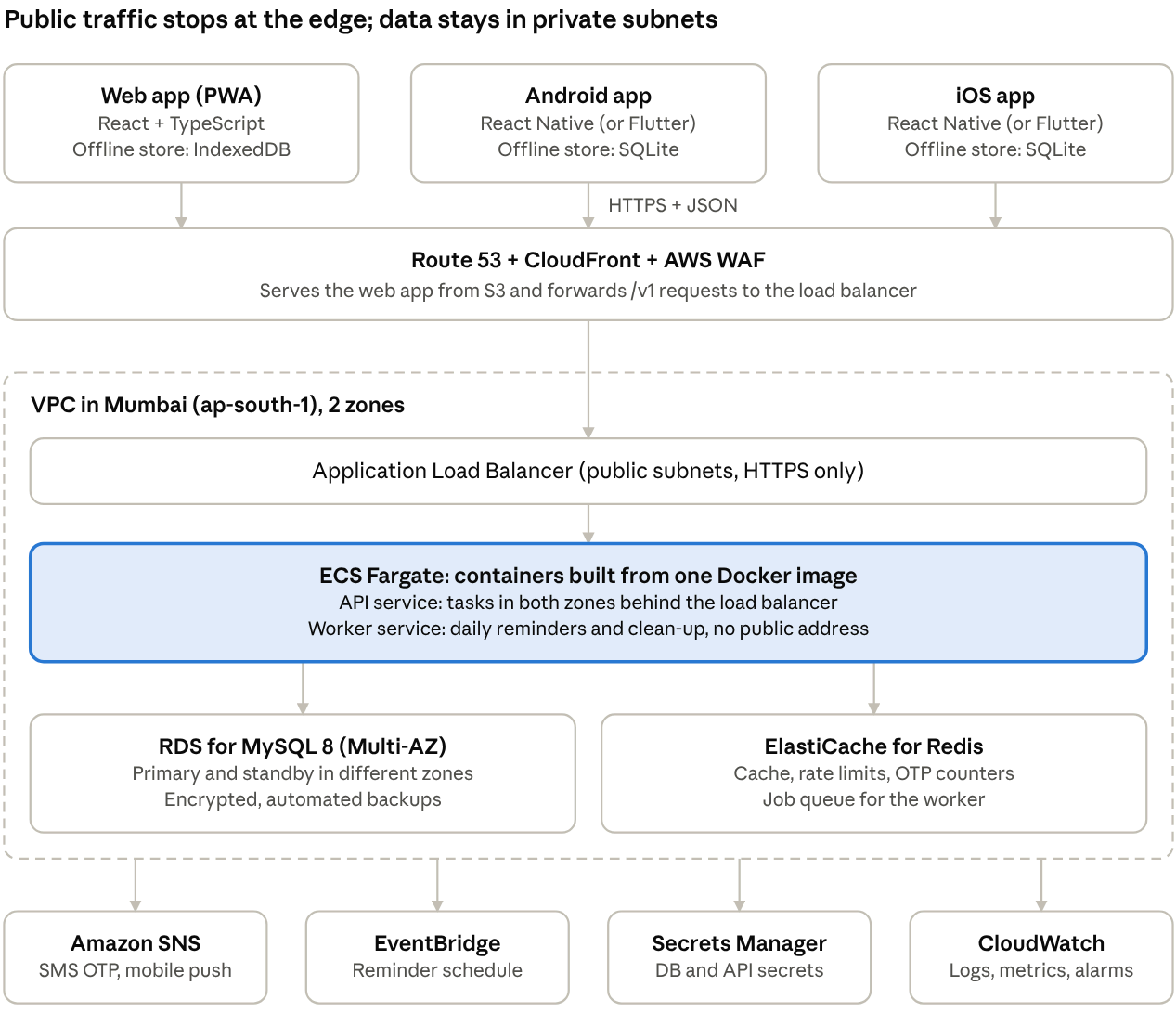
# Simple Expense Tracker – System Architecture (AWS)

Oct 13, 2026 · @skr

## 1. Architecture at a glance




Managed AWS services called by the API and worker

AWS architecture · edge, VPC, compute, data, managed services

Production runs on AWS in the Mumbai region (ap-south-1). Route 53, CloudFront and AWS WAF are the only public entry. They serve the web app from S3 and pass `/v1` requests to an Application Load Balancer. The API runs as containers on ECS Fargate in private subnets and talks to Amazon RDS for MySQL and ElastiCache for Redis. Each client keeps its own local copy of the data, so the app keeps working offline and syncs later.

On your laptop, the same app runs with MySQL (installed locally and used through MySQL Workbench) and Redis in Docker Compose, so local and production behave the same.

## 2. Components and AWS services

The design is deliberately plain: one API service, one MySQL database, one cache. This is enough for the first million users and keeps the team small.

| Component | Responsibility | Technology / AWS service |
|---|---|---|
| Web app | All screens, offline storage, quick-add chips, charts. Installable as a PWA. | React, TypeScript, Vite; built files in S3, delivered by CloudFront |
| Mobile apps | Same screens and logic as web; local SQLite; reminders; push tokens. Built in phase 3. | React Native (or Flutter) |
| Local store | Holds expenses, chips and limits on the device; marks rows as pending until synced. | IndexedDB (web), SQLite (mobile) |
| DNS, CDN, firewall | Domain, HTTPS certificate, caching of the web app, blocking of bad traffic and abuse. | Route 53, CloudFront, AWS Certificate Manager, AWS WAF |
| Load balancer | Spreads API traffic across containers in two zones; terminates HTTPS. | Application Load Balancer |
| REST API | Validates requests, applies business rules (auto-category, limits, reports), enforces ownership. | Node.js with NestJS, packaged as a Docker image in Amazon ECR, run on ECS Fargate |

| Component | Responsibility | Technology / AWS service |
|---|---|---|
| Database | Permanent storage of all user data, as in the Database Design doc. | Amazon RDS for MySQL 8.0, Multi-AZ, encrypted with KMS |
| Cache and queue | Rate-limit counters, OTP attempt counters, short-lived cache of dashboard results, job queue. | Amazon ElastiCache for Redis |
| Background worker | Sends daily reminders, purges soft-deleted rows after 90 days, expires old OTPs and tokens. | Same Docker image as the API, run as a second ECS service; EventBridge Scheduler starts the daily jobs |
| SMS and push | Delivers the login OTP by SMS, and reminders and limit alerts as push messages (phase 2). | Amazon SNS (SMS needs DLT registration for India); SNS mobile push to FCM and APNs |
| Google sign-in | Verifies a Google ID token. | Google Identity Services |
| Secrets and config | Database password, JWT signing key, SMS and Google credentials. | AWS Secrets Manager, injected into the containers |
| Logs and alerts | Application logs, metrics, alarms. | Amazon CloudWatch; error tracking with Sentry |

**Why one service, not microservices.** The app has six small modules that share one database. Splitting them would add network calls, deployment work and failure points with no benefit at this size. The modules in section 4 keep clear boundaries, so a module can be split out later if needed.

**Why ECS Fargate.** It runs containers without managing servers, scales by adding tasks, and is simpler than Kubernetes (EKS) for a small team. If you want the very simplest start, AWS App Runner or Elastic Beanstalk can run the same container, but they give less control over private networking, so this doc uses ECS.

**Why RDS for MySQL.** It gives the same MySQL 8 you use in Workbench, with backups, patching, failover and encryption handled by AWS. Aurora MySQL is a drop-in upgrade later if you need more read capacity.

**Why offline-first.** Users log expenses on buses and in markets with weak signal. Saving locally first makes every add instant and removes the "no network" failure from the main flow.

# 3. Key flows

### Add an expense (offline-first)

1. The user types "Auto" and 10, then taps Add (or taps the Auto chip).
2. The app creates a UUID, saves the row in the local store and marks it pending. The home screen updates at once.
3. If the user is signed in and online, the app calls `PUT /expenses/{id}`.
4. The API validates the request, guesses the category from the item name, saves the row, updates the quick-add chip, recalculates limit progress and returns `limit_status`.
5. The app marks the row synced and shows a gentle message if a limit crossed 80% or 100%.
6. If the call fails (no network, server error), the row stays pending and is retried later with the same ID, so no duplicate is created.

### Sign in and first sync

1. The user taps "Back up my data" and enters a phone number.
2. The API sends an OTP through Amazon SNS; Redis limits requests to 5 per hour per phone.
3. The user enters the code. The API returns an access token (15 minutes) and a refresh token (30 days).
4. The app uploads all local rows with `POST /sync/push`, then fetches anything newer with `GET /sync/changes`.
5. From then on, every change syncs in the background, and a new device gets the full history after sign-in.

### Weekly report and dashboard

1. The app asks for `GET /reports/weekly` or `GET /dashboard`.
2. The API checks Redis for a cached result for that user and period.
3. On a miss it runs the report queries on MySQL, builds the response and caches it for a short time.
4. Any expense change for that user clears their cached reports, so numbers are never stale.
5. Guests get the same report calculated on the device from the local store, using the same rules.

### Daily reminder (phase 2)

1. Each evening, EventBridge Scheduler starts the reminder job in the background worker, which finds users whose reminder time has arrived in their time zone and who have not logged an expense today.
2. It sends a push message through Amazon SNS mobile push (FCM or APNs). Web reminders use the browser's notification permission if the user grants it.

## 4. Code structure and sharing

Use one repository (monorepo) so the web app, mobile apps and API share types and rules.

```
expense-tracker/
  apps/
    api/          NestJS service + background worker entry point
    web/          React PWA
    mobile/       React Native app (phase 3)
  packages/
    shared/       TypeScript types, validation rules, money and date helpers,
                  limit and report calculations, category keyword list
    api-client/   Typed client generated from the OpenAPI file
    ui/           Shared design tokens (colours, spacing, text sizes)
  infra/          AWS setup as code (Terraform or CDK), Dockerfile, MySQL
migrations
  docs/           PRD, API design, DB design, architecture
```

**API modules** (one folder each, each with controller, service and repository): `auth`, `expenses`, `categories`, `limits`, `reports`, `sync`, `users`. Rules for the modules:

- A module reaches another module only through its service, never its tables.
- Business rules (category guess, limit percentage, week boundaries) live in `packages/shared`, so the offline guest mode on the device and the server give identical answers.
- Every request is checked against the signed-in user's ID, so one user can never read another user's rows.
- Controllers only validate input and call services; no SQL outside repositories.

**Web and mobile sharing.** The web app and the mobile app share the shared package, the API client and the design tokens. Screens are written separately for each platform, but they use the same hooks (for example `useAddExpense`, `useWeeklyReport`), so behaviour matches everywhere.

## 5. AWS deployment, scaling and monitoring

**Environments.** Three: local, staging and production. Local runs MySQL on your laptop (used through Workbench) and Redis in Docker Compose. Staging and production are separate AWS environments, each with its own VPC, database and secrets. Keep them in separate AWS accounts under AWS Organizations if you can; at minimum use separate VPCs.

**Network layout (per environment, region ap-south-1)**

- One VPC across two Availability Zones with three kinds of subnet: public (load balancer and NAT gateway), private application (ECS tasks), private data (RDS and ElastiCache).
- Security groups pass traffic only one step at a time: the load balancer accepts HTTPS only from CloudFront; ECS tasks accept traffic only from the load balancer; RDS accepts port 3306 only from the ECS tasks; Redis accepts only from the ECS tasks.
- The database and Redis have no public address. For admin work in MySQL Workbench, use an SSH tunnel through a small bastion instance or an AWS Systems Manager port-forward.
- ECS tasks reach SNS, Secrets Manager and Google through the NAT gateway; VPC endpoints can replace some of it later to cut cost.

**Build and release**

1. A pull request runs lint, unit tests and API tests against a throwaway MySQL container (for example in GitHub Actions).
2. Merging to main builds the Docker image and pushes it to Amazon ECR.
3. The pipeline runs database migrations as a one-off ECS task, then deploys the new image to staging with a rolling update and health checks.
4. After a manual approval, the same image is deployed to production. ECS keeps the old tasks running until the new ones are healthy, and the deployment circuit breaker rolls back automatically if they fail.
5. The web app is built, uploaded to S3, and the CloudFront cache is cleared. Mobile apps go through Google Play and the App Store with staged rollouts.

6. Migrations must be backward compatible with the previous app version, so a rollback never meets a schema it cannot read.

**Scaling stages.** Scale only when the numbers call for it.

| Stage | AWS setup | Move to the next stage when |
|---|---|---|
| Launch | 2 API tasks (one per zone), 1 worker task, one small RDS MySQL instance with Multi-AZ, one small ElastiCache node, CloudFront + S3 for the web app. Staging can use single-AZ to save cost. | API CPU stays high at peak hours |
| Growth | ECS auto scaling on CPU and request count, a larger RDS instance class, RDS Proxy for connection pooling, an RDS read replica used for reports | Report queries slow down the main database, or connections run short |
| Scale | Aurora MySQL, partition or archive `expenses`, separate worker pool, longer-lived report cache | Only if usage demands it |

The API keeps no state in memory (sessions are tokens, counters are in Redis), which is what lets it run as many copies as needed.

### Monitoring and backups

- Logs from all containers go to CloudWatch Logs. Alarms on load balancer 5xx errors and response time, ECS CPU and memory, RDS CPU, free storage and connections, and ElastiCache memory notify the team through SNS (email or chat).
- Error tracking (for example Sentry) on web, mobile and API.
- RDS automated backups with point-in-time recovery, kept for at least 7 days, plus a manual snapshot before each production migration. Restore a backup into staging regularly to prove it works.
- AWS CloudTrail on, so every change to the AWS account is recorded.
- A status check endpoint ( `GET /health` ) is used by the load balancer to decide which tasks are healthy.

## 6. Security, build order and cost

### Security and privacy

- HTTPS everywhere with free ACM certificates; HSTS on the web app and API.
- AWS WAF in front of CloudFront with AWS managed rule groups and a rate-based rule per IP.
- IAM roles for the ECS tasks follow least privilege: each task can read only its own secrets and publish to SNS. No AWS access keys in code or in the app. Turn on MFA for the root user and give people access through IAM Identity Center.
- Database password, JWT signing key and third-party keys live in Secrets Manager; rotate the database password on a schedule.
- RDS storage and backups are encrypted with KMS, and the API connects to MySQL over TLS. The app's MySQL user has only the rights it needs; migrations use a separate user.
- The S3 bucket is private; only CloudFront can read it.
- Short-lived access tokens (15 minutes) and rotating refresh tokens stored as hashes. On the web, keep the refresh token in an HttpOnly cookie; on mobile, in the Keychain or Keystore.
- OTP codes expire after 5 minutes, lock after 5 wrong tries and are rate-limited per phone and per IP.
- Every query filters by the signed-in user's ID. Tests must prove one user cannot read or change another's data.
- Data stays in the Mumbai region. Consent screen at sign-up, data export (CSV) and full account deletion, in line with India's Digital Personal Data Protection Act.
- No ads, no third-party trackers, and no sale of user data.

**Build order**

1. Shared package: money, dates, category keywords, limit and report calculations.
2. API with MySQL: create the schema in Workbench, then turn it into migration files. Expenses, categories, limits and reports first; guest data lives on the device.
3. Web app: add screen, chips, history, limits, weekly report, dashboard, offline store. This is the web MVP in the PRD, running fully on your laptop.
4. AWS foundation in staging, written as code: VPC, RDS, ElastiCache, ECR, ECS, load balancer, CloudFront, S3. Deploy the web MVP there and test with real phones.
5. Production launch of the web app on AWS, then auth, sync, reminders and Hindi (PRD phase 2).
6. Mobile apps on the same API and shared package, Android first and then iOS (PRD phase 3).

Because the rules live in the shared package and the API is the same for every client, step 6 is mostly screens, not new logic.

**Cost drivers.** The biggest steady AWS costs are usually the RDS Multi-AZ database, the NAT gateway, the load balancer, the ElastiCache node and the always-on Fargate tasks. Costs that grow with users are SMS for OTP logins through SNS, data transfer and CloudFront requests. To keep the bill low: use single-AZ in staging, start with small instance sizes, set AWS Budgets alerts from day one, offer Google sign-in as a cheaper alternative to SMS, and consider Savings Plans or reserved instances once usage is steady. Estimate the real numbers with the AWS Pricing Calculator; this doc gives no price figures.